АРХИТЕКТУРНЫЙ АУДИТ САЙТА · НОЯБРЬ 2026

СканРецепт

Проверен весь код, данные и фото-конвейер: приложение, 399 рецептов, 260 фотографий, деплой на два зеркала. Каждая находка объяснена простыми словами — без технического жаргона.

Отчёт для владельца проекта

28 находок: 4 критичных · 10 важных · 14 полировка

Z.AI · АУДИТ ARCH-1 · 05.11.2026

Содержание

1. Зачем этот отчёт и как его читать 1

2. Как устроен сайт 2

3. Что уже в порядке (зелёные зоны) 3

4. Критичные проблемы — 4 (чиним сразу) 3

5. Важные проблемы — 10 (планируем на недели) 4

6. Полировка — 14 (когда будет время) 5

7. Как проверяются фотографии 6

8. План действий: 10 быстрых побед 8

9. Словарик терминов 9

1. Зачем этот отчёт и как его читать

5 ноября 2026 года сайт СканРецепт прошёл полный «техосмотр»: проверен весь код приложения, все данные о блюдах, все 260 фотографий, скрипты публикации и даже то, как сайт выглядит для поисковиков. Сама еда не проверялась — проверялось то, **как устроен сайт внутри**, как автомобиль на СТО: смотрят не на цвет кузова, а под капот. Такой осмотр называется **архитектурным аудитом**.

Всего набралось **28 находок**. Каждая отнесена к одному из трёх уровней серьёзности. **Критичные** — их чиним сразу: это утечки информации и поломки в главной функции сайта. **Важные** — они не ломают сайт сегодня, но заметно замедляют его и мешают развитию; их стоит распланировать на ближайшие недели. **Полировка** — мелкие недочёты и «мусор» в коде: наводим порядок, когда будет свободный час.

260	399	28	0
фотографий блюд	рецептов в данных	находок аудита	ошибок проверки типов

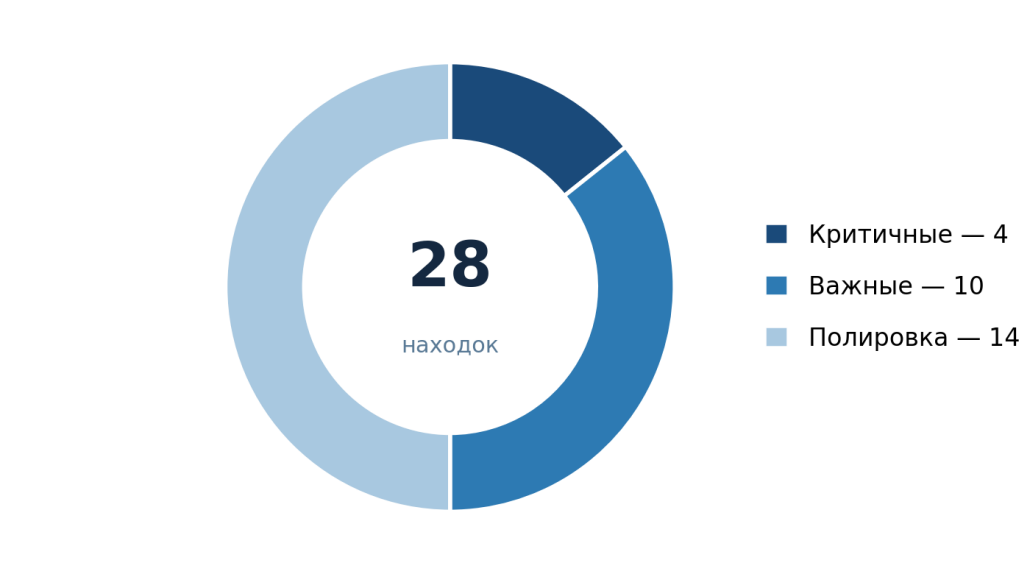


Рисунок 1. Все 28 находок по уровням серьёзности: 4 критичных, 10 важных, 14 полировка

Главный вывод. Сайт в хорошей форме: данные о блюдах образцовые (ни одного дубликата или битой ссылки), замок платной части непробиваем, ошибок в типах — ноль. Реальные риски сосредоточены в трёх местах: публикация лишних файлов, работа со сканером штрих-кодов и вес «стартового» файла приложения. Всё это чинится, и почти всё — быстро.

2. Как устроен сайт

СканРецепт — это **PWA** (Progressive Web App): обычный сайт, который можно «установить» на телефон как приложение и который частично работает без интернета. Схема ниже показывает весь путь: где сайт живёт, как к нему попадает посетитель и что приложение использует для работы.



Рисунок 2. Большая картина: из чего состоит СканРецепт и как узлы связаны между собой

Хостинг. Сайт размещён на бесплатном хостинге GitHub Pages и доступен по двум адресам-«зеркалам»: главному hopmyl.github.io и запасному hopmyl.github.io/scanfood/. Зеркала — это две копии одного и того же сайта: если одна недоступна, работает вторая. Обновляются они одним скриптом deploy-all.sh.

Приложение. Весь код (7 экранов, 3 компонента, общее хранилище настроек) собирается в один файл index-*.js — его называют **бандл** («свёрток»). Сейчас он весит около 1,3 МБ, и посетитель скачивает его целиком при первом открытии — об этом в разделе 8.

Данные и фото. Рецепты хранятся прямо в коде — 399 блюд в 25 файлах-таблицах; 260 из них уже имеют фотографии, и только они показываются в каталоге (161 бесплатных ЛАЙТ и 99 платных PRO). **Сканер** — единственная часть, которая ходит во внешние сервисы: снимок продукта уходит нейросети OpenRouter на распознавание, а штрих-коды сверяются с открытой базой Open Food Facts.

3. Что уже в порядке (зелёные зоны)

Аудит — это не только список проблем. Не меньше половины проверки пришлась на зоны, которые работают отлично. Их важно знать: при будущих доработках эти части нельзя случайно сломать, а переделывать «на всякий случай» — не нужно.

Зона	Что именно проверено и в порядке
Данные о блюдах	0 дубликатов id; каждая фотография на месте (260 из 260); платные/бесплатные списки без ошибок; калории сходятся с формулой 4/4/9; дублей названий нет
Платный доступ (пейволл)	PRO-замок в карточке блюда непробиваем из интерфейса; скан списывается до распознавания и честно возвращается при неудаче; демо-скан не списывает лимит
Сканер	правильно понимает все коды ошибок: 429 показывает человеческим текстом, отключённый сервис — «недоступно», а не пугающим сбоем; возвраты и «Повторить» работают
PWA и кэш	кэшируется только «каркас» приложения, фото — нет; старые версии автоматически удаляются; иконки и manifest корректны
Мобильная версия	учтены «чёлки» и жести iPhone, тап-подсветка отключена, темы меняются без перезагрузки
SEO-база	русский язык страницы, заголовки, описания картинок — всё на месте
Сборка и деплой	адреса для двух зеркал собираются корректно; инструкции для Vercel, Netlify и РФ-хостингов в порядке
Типы и качество кода	строгий режим TypeScript: 0 ошибок, ни одного обхода типов

Таблица 1. Зелёные зоны: что проверено и работает как надо

4. Критичные проблемы — 4 (чиним сразу)

Это находки, которые либо уже привели к утечке, либо ломают главную функцию сайта. Формулировка у каждой одна и та же: что случилось, почему это важно и что делать.

К1. Архив исходников лежал в публичной папке

В публичной папке сайта находился архив skanrecept-src-2026-10-29.zip (15 МБ, 161 файл) со всем кодом и всей рецептурной базой — главным активом проекта. Любой посетитель мог скачать его одной ссылкой, а при следующем обновлении архива туда легко попал бы и настоящий секретный ключ. **Статус: уже исправлено** — архив удалён и добавлено правило против попадания новых zip-архивов в публичную папку (коммит b1fca3f).

К2. Ключ распознавания вшивается в файл приложения

При сборке сайта для зеркал в открытый файл приложения зашивается ключ сервиса OpenRouter. Технически любой посетитель может вытащить его из браузера. Сейчас это не страшно: ключ без денег на счету и работает только с бесплатными моделями. Но защита должна быть дисциплиной, а не везением: держать на ключе нулевой кредитный лимит, следить за расходом и ротировать его; в перспективе — убрать ключ из публичного файла совсем и ходить через свой сервер-посредник.

К3. Сканер штрих-кодов «замирает» после первого товара

После первого найденного (или не найденного) товара камера останавливается, но экран выглядит как живой: видоискатель и лазерная линия на месте, а кадр заморожен и сканирование не идёт. Продолжить можно только переключив режим «Продукты → Штрих-код» и обратно. Это ломает главный сценарий «отсканировать несколько покупок подряд» — ядро всей функции. Лечение: кнопка «Сканировать другой» должна заново запускать камеру (перезапуск эффекта через счётчик запусков).

К4. При зависшем сервере посетитель ждёт до двух минут

Распознавание продукта идёт по цепочке из трёх серверов. Если первый в цепочке жив, но жутко тормозит, клиент честно ждёт каждый шаг: 55 + 55 + 70 секунд — до трёх минут с надписью «Распознаём продукты…». На телефоне столько никто не подождёт. Лечение: разные лимиты ожидания для разных шагов (10–25 секунд вместо 55) и запуск запросов «наперегонки» — чей ответ пришёл первым, тот и побеждает.

5. Важные проблемы — 10 (планируем на недели)

«Важные» — это не аварии, а то, что тормозит сайт и мешает развитию: тяжёлый стартовый файл, копипаста в четырёх местах, вечное накопление мусора на зеркалах. Каждая строка таблицы — суть проблемы и быстрое решение; детали — в исходном отчёте architect-report.md.

№	Проблема и почему важно	Быстрое решение
В1	Стартовый файл — монолит 1,84 МБ: библиотека сканера штрих-кодов (zxing, 0,5 МБ) скачивается всеми, даже тем, кто никогда не сканирует	ленивая загрузка: подгружать zxing и тяжёлые экраны только при входе в режим
В2	Прямой запрос к базе товаров без таймаута: при сбое кнопка «Ищем товар» висит вечно	таймаут 9 секунд, проверка ответа, кэш товара в браузере
В3	Логика распознавания скопирована в 4 копии (Vercel, Netlify, сервер, клиент): правка в одном месте расходится с остальными	один общий источник + автопроверка совпадения копий при сборке

№	Проблема и почему важно	Быстрое решение
B4	Старые версии файлов навсегда копятся на зеркалах: репозиторий публикации растёт с каждым деплоем	зеркалирование через rsync --delete: лишнее удаляется
B5	Страница 404.html (спасает прямые ссылки на телефонах) создаётся только руками в скрипте деплоя — новый способ публикации молча её потеряет	перенести копирование в шаг сборки сайта
B6	Не было превью для мессенджеров: ссылка в Telegram/WhatsApp разворачивалась без картинки — а «вкусная картинка» приводит установки	**уже добавлено** (OG-теги, ноябрь 2026)
B7	robots.txt без карты сайта; два зеркала конкурируют между собой в поиске	добавить sitemap.xml из 260 карточек и canonical-ссылку на главное зеркало
B8	На платной карточке рисуются два одинаковых бейджа PRO один поверх другого	удалить дубль, оставить один бейдж
B9	Надпись «осталось 5 сканов» повторяет цифру в трёх местах: поменяешь лимит — текст соврёт	везде использовать константу FREE_SCANS_LIMIT
B10	Устаревшие числа в комментариях и README («54 страны» вместо 111, «100 рецептов» вместо 260/399)	пересчитать и переписать; выдавать фактические квоты скриптом

Таблица 2. Важные находки B1–B10

6. Полировка — 14 (когда будет время)

Эти находки не влияют на работу сайта сегодня. Это «гигиена кода»: маленькие шероховатости, которые приятно закрыть одним вечером наведения порядка.

№	Заметка	Что сделать
H1	Выбранное число порций не сбрасывается при переходе к другому блюду	сбрасывать при смене блюда
H2	Защита сканера от двойного запуска фактически не работает (устаревшее замыкание)	флаг-переключатель через useRef
H3	46 ингредиентов в 36 блюдах без количества («Соль, перец»)	добавить пометку «по вкусу»
H4	Подбор «добавь один продукт» не знает части смесей («Имбирь, чеснок»)	дополнить справочник продуктов
H5	Список «блюда без фото» вычисляется, но нигде не показывается	показать счётчик или удалить
H6	Блюдо без описания молча исчезнет из фильтра «до 30 минут»	проверка при сборке

№	Заметка	Что сделать
H7	Каталог рисует все 260 карточек сразу — на слабых телефонах ощутимо	показывать по 40 карточек
H8	Счётчик сканирований в профиле не откатывается при возврате скана	решить и задокументировать
H9	Текст «сервер не ответил за 55 с» пришит к таймауту намертво	подставлять число из константы
H10	Токен деплоя вырезается из адреса репозитория — хрупко	брать из переменной окружения
H11	260 фото (33 МБ) перезаливаются на зеркала при каждом деплое	сжать в WebP + подключить CDN
H12	Не загрузившееся фото прячется в заглушку до перезагрузки страницы	одна повторная попытка
H13	60+ служебных файлов фото-конвейера лежат в scripts/	перенести в ops/photo-pipeline/
H14	Тяжёлые папки (скрины, архивы, дампы) коммитятся в репозиторий	добавить в .gitignore

Таблица 3. Полировка H1–H14

7. Как проверяются фотографии

У каталога 260 фотографий, и все они генерируются нейросетью. Чтобы посетитель не видел «пластиковых» или выдуманных блюд, работает автоматический конвейер качества: он не только создаёт фото, но и сам же их проверяет и переделывает брак.

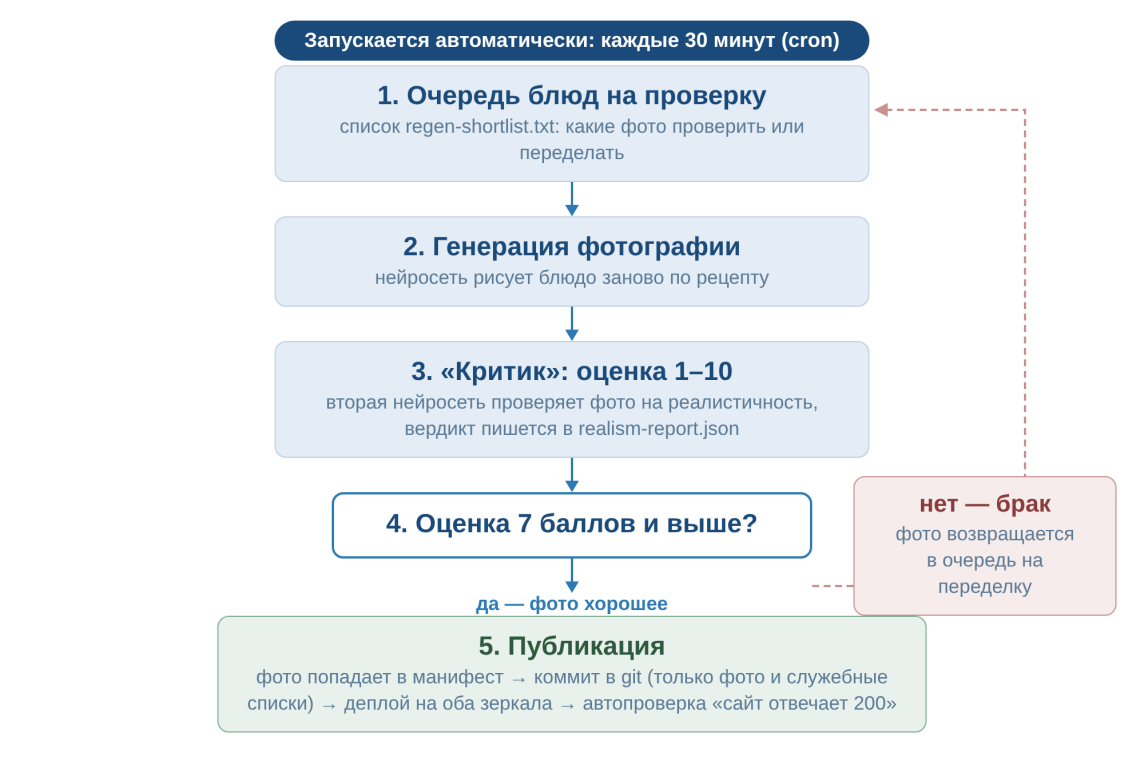


Рисунок 3. Конвейер проверки фотографий: от очереди блюд до публикации на зеркалах

Конвейер запускается автоматически каждые 30 минут (это называется **крон** — задача-будильник на сервере). Он берёт блюда из очереди, генерирует фото, отправляет его второй нейросети-«критику», и только оценка 7 баллов и выше отправляет фото публикацию. Брак возвращается в очередь на переделку — сколько бы попыток ни потребовалось.

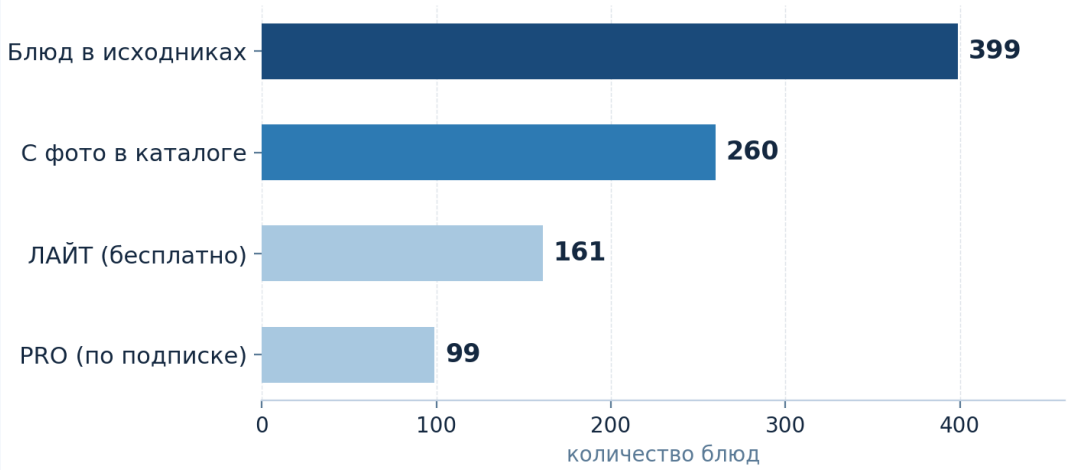


Рисунок 4. Каталог в цифрах: 399 блюд в данных, 260 с фото, из них 161 ЛАЙТ и 99 PRO

Свежий результат. Шесть фотографий, которые вы отметили как старые, уже переделаны и одобрены «критиком»: стейк — 9 из 10, карбонара — 8, рёбрышки BBQ — 8, бургер — 7, соба с говядиной — 7, спагетти болоньезе — 7. Очередь старых фото пуста.

Про тариф Lite. Генерация фотографий идёт через платный сервис, и ещё 3 ноября конвейер упирался в «429-штормы»: сервер отвечал «слишком много запросов», генератор ставился на паузу, а прогоны проходили вхолостую (это видно в журнале heartbeat.log). Оплаченный тариф Lite поднимает квоты и лимиты запросов — значит, «штормов» станет меньше, а очередь пройдёт быстрее. Прогоны 4–5 ноября уже отработали без отказов: очередь старых фото закрыта полностью. Менять в конвейере ничего не нужно — он просто реже упирается в потолок.

8. План действий: 10 быстрых побед

Ниже — те же находки, но отсортированные по принципу «максимум эффекта за минимум времени». Первые пять пунктов занимают около полутора часов суммарно и закрывают и утечку, и главный тормоз сайта, и сломанный сценарий сканера.

№	Действие	Время	Эффект
1	Удалить публичный архив исходников и запретить zip в public/	5 мин	**сделано 05.11** — утечка закрыта, минус 15 МБ с каждого деплоя
2	Ленивая загрузка библиотеки штрих-кодов (zxing)	15 мин	минус 0,5 МБ из стартового файла
3	Ленивая загрузка тяжёлых экранов (React.lazy + Suspense)	30 мин	каталог открывается без сканера и лишнего кода
4	Починить возобновление сканера после результата (К3)	30 мин	работает «несколько покупок подряд»
5	Таймаут 9 с + проверка ответа + кэш в прямом запросе товаров (В2)	20 мин	штрих-код больше не зависает навечно
6	OG-теги + иконка iOS + картинка-превью	20 мин	**сделано 05.11** — красивое превью в мессенджерах
7	Свести промпт распознавания к одной копии + проверка хешей	1–2 ч	конец классу багов «на одном сервере одно, на другом другое»
8	Дифференцировать таймауты цепочки скана (К4)	10 мин	худшее ожидание: 180 с → около 90 с
9	Убрать двойной PRO-бейдж; тексты «5 сканов» из константы; обновить числа	20 мин	честные тексты и чистый код
10	rsync --delete в деплое + 404.html в сборке	20 мин	зеркала не распухают, ссылки не молча ломаются

Таблица 4. Топ-10 быстрых побед: время указано чистое, на человека

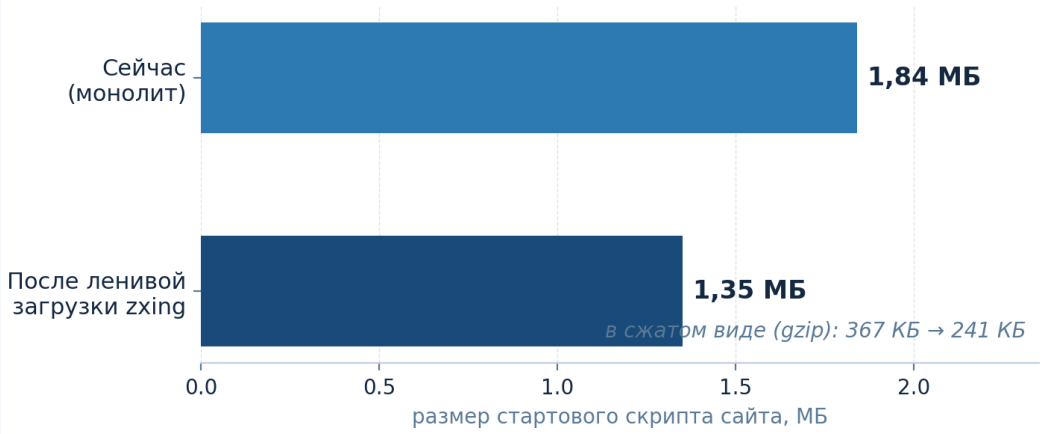


Рисунок 5. Что даёт ленивая загрузка сканера: стартовый файл худеет на четверть

Рекомендуемый порядок: пункты 2–5 одним заходом (это один вечер, они все крутятся вокруг сканера), затем пункт 7 — он убирает целый класс будущих багов. Пункты 9–10 удобно закрыть тем же вечером «полировки» из раздела 6.

9. Словарик терминов

Все термины из этого отчёта — в одну строчку, без формальностей.

Термин	Что это значит простыми словами
PWA	сайт, который можно «установить» на телефон как приложение и который частично работает без интернета
Бандл	один файл, в который собран весь код сайта; его скачивает браузер при первом открытии
Gzip	сжатие файлов при передаче: тот же бандл, но «запакованный» — легче в 4–5 раз
Ленивая загрузка	код подгружается не сразу, а когда реально понадобился: сканер — только при входе в режим сканирования
Зеркало	полная копия сайта по другому адресу — страховка на случай сбоя
SPA	приложение-страница: сайт без перезагрузок, все экраны живут в одном файле
404.html	заглушка, благодаря которой прямая ссылка (например, на блюдо) открывается корректно, а не страницей ошибки
OG-теги	описание и картинка, которые показывают мессенджеры, когда кто-то делится ссылкой на сайт
Sitemap	карта сайта для поисковиков: список всех страниц, чтобы Google ничего не пропустил
Canonical	пометка «главный адрес» — чтобы два зеркала не конкурировали в поиске
Таймаут	максимальное время ожидания ответа: «жду 9 секунд, дальше — не жду»

Термин	Что это значит простыми словами
Крон	задача-будильник на сервере: запускаться сама, например, каждые 30 минут
Кэш	копия файлов в браузере или на телефоне, чтобы не скачивать заново
Коммит	сохранённая версия изменений в репозитории (хранилище кода)
Деплой	публикация: выложить свежую версию сайта на хостинг

Таблица 5. Словарик: каждый термин одной фразой

Что дальше. Сайт в рабочей форме, критические риски взяты под контроль: утечка закрыта, превью для мессенджеров добавлено, фото-конвейер вычищает брак сам. Следующий логичный шаг — «быстрые победы» №2–№5 из раздела 8: один вечер работы, а сайт станет заметно быстрее и надёжнее. Скажите — и начнём с них.